

北京邮电大学

自然语言处理

课程实践结题报告



题目： 来易卦，来一卦——基于古汉语检索增强
生成的易学咨询智能体

学 号： 2022211733 2022211735 2022211799

姓 名： 王殿云 王博涵 王艺栋

学科专业： 人工智能

班 级： 2022219107 2022219108

学 院： 人工智能学院

2025 年 6 月 19 日

摘 要

古汉语文本的语义理解是自然语言处理领域的重要挑战，其核心难点在于古今汉语之间存在显著的语义鸿沟。本文提出了一种基于检索增强生成的古汉语智能问答系统，通过融合语言学知识与深度学习技术，实现了高质量的跨时语义匹配。我们的主要贡献包括：首先，提出了一种基于双音节对齐替换的向量表示微调方法，该方法利用汉语演变的语言学规律，结合难负例挖掘技术，在 C³Bench 基准测试上将跨时语义检索的 Recall@1 从 89.17% 提升至 93.44%。其次，构建了包含 28 部易学典籍、超过 300 万字符的层次化知识库，采用年代-领域-书目-文本的四级索引结构，支持多粒度的语义检索。最后，设计并实现了遵循 Dify 插件协议的检索增强生成工具，并基于 ReAct 架构构建了易学咨询智能体。实验结果表明，我们的方法在保持计算效率的同时，显著提升了古汉语语义理解的准确性。系统已通过微信小程序形式部署上线，在两周内积累了 8000 余名用户，验证了技术方案的实用性。



目 录

1. 引言	5
2. 相关工作	5
2.1 古汉语处理技术	5
2.1.1 古汉语语言学特征与计算挑战	5
2.1.2 面向古汉语处理的大语言模型	6
2.2 检索增强生成技术	6
2.3 智能体框架与工具集成	7
2.3.1 ReAct 智能体框架	7
2.3.2 Dify 大语言模型应用开发平台	7
2.4 古汉语语料资源与评估基准	7
3. 方法	7
3.1 问题定义	7
3.2 难负例构造技术	8
3.3 双音节对齐替换正例构造技术	8
3.4 混合策略与训练框架	9
3.5 理论分析	9
3.6 层次化知识库构建	10
4. 实验	10
4.1 实验设置	10
4.1.1 数据集与预处理	10
4.1.2 实验配置	10
4.1.3 评估指标	11
4.2 主要结果	11
4.2.1 训练过程分析	11
4.2.2 整体性能对比	12
4.2.3 分类别性能分析	14
4.2.4 嵌入空间可视化分析	15
4.2.5 错误分析与案例研究	16
4.3 实验结论	19
5. 应用与部署	19
5.1 系统架构设计	19
.....	19
5.2 遵从 DIFY 协议的插件设计	20
5.2.1 YiGuaRAGServer——支撑远端调用的多级索引 RAG 服务器	20
5.2.2 YiGuaRAGTool——支撑 Dify 工具调用协议的多级索引 RAG 模块	21
5.2.3 YiGuaAgent——基于 ReAct 架构的易学 Agent 框架	22
5.3 易卦 AI——易学咨询小程序	23

5.3.1 主要功能	23
5.3.2 界面展示	23
5.4 部署方案	24
5.4.1 后端服务容器化部署	24
5.4.2 前后端分离架构实现	24
6. 结论	25
参考文献	26
附录	26
附录 A: 正负例构造技术关键实现	26
A.1 双音节词对齐算法	26
A.2 多粒度正例生成	29
A.3 基于 Embedding 的难负例挖掘	30
A.4 训练数据整合	31
附录 B: 与 MRASP 方法的等价性完整证明	34
B.1 问题设定与符号定义	34
附录 C: 系统组件关键实现细节	37
C.1 YiGUARAGSERVER 项目结构	37
C.2 多级索引查询实现	38
C.2.1 书目检索 API 实现	38
C.2.2 数据库查询逻辑	39
C.3 两阶段检索实现	40
C.3.1 向量检索实现	40
C.3.2 重排序实现	42
C.4 文言文文本处理器实现	43
C.5 YiGUARAGTOOL 插件结构	45
C.6 智能参数处理实现	45
C.6.1 工具提供者凭证验证	45
C.6.2 参数解析与处理	47
C.7 多格式输出处理实现	48
C.8 错误处理实现	49
C.9 YiGUAAGENT 典型交互示例	51
C.9.1 单轮检索示例	51
C.9.2 多轮深入检索示例	51
附录 D: 成员分工	54

1. 引言

随着数字人文的快速发展，如何让机器准确理解和处理古代文献成为自然语言处理研究的前沿课题。古汉语作为中华文明的重要载体，其数字化和智能化处理具有重要的学术价值和应用前景。然而，古汉语文本的计算理解面临着独特的挑战：首先，古汉语以单音节词为主，而现代汉语中双音节词占主导地位，这种词汇结构的系统性差异导致传统的语言模型难以建立有效的语义映射；其次，同一概念在不同历史时期具有完全不同的表达方式，语法结构和用词习惯的演变进一步加剧了跨时语义理解的复杂性；最后，现有的预训练语言模型主要基于现代文本训练，在处理古汉语时存在严重的领域偏移问题。

近年来，检索增强生成技术在知识密集型任务中展现出巨大潜力，为古汉语智能问答提供了新的技术路径。然而，将 RAG 技术直接应用于古汉语场景仍面临诸多挑战：现有的文本嵌入模型在跨时语义匹配上表现不佳，无法准确检索语义相关但表达迥异的古今文本对；缺乏针对古籍特点设计的知识组织和索引机制；现有的智能体框架难以处理古汉语理解所需的多轮推理和知识整合。

本文针对上述挑战，提出了一套完整的古汉语检索增强生成解决方案。我们的核心创新在于设计了一种基于语言学知识的向量表示学习方法，通过显式建模古汉语到现代汉语的演变规律，实现了高质量的跨时语义对齐。具体而言，我们提出了双音节词对齐替换技术，利用动态规划算法自动发现古文单字与现代双音节词的对应关系，并通过概率性替换生成多粒度的训练样本。同时，我们采用基于嵌入相似度的难负例挖掘策略，强化模型的细粒度语义区分能力。理论分析表明，该方法与多语言预训练领域的 mRASP 技术在表示学习上具有等价性，但计算效率更高。

在系统实现层面，我们构建了覆盖多个朝代和领域的易学古籍知识库，设计了适应古籍特点的文本处理和索引机制。通过遵循 Dify 插件开发协议，我们实现了模块化的检索增强生成工具，并基于 ReAct 架构构建了能够进行多轮推理的智能体系统。该系统不仅在学术评测上取得了优异成绩，还通过实际应用验证了其商业价值，为古汉语智能处理的产业化提供了可行路径。

2. 相关工作

2.1 古汉语处理技术

2.1.1 古汉语语言学特征与计算挑战

古汉语处理为易学文献的计算理解提供了技术基础。早期研究主要采用统计方法应对古汉语特有的语言学挑战，如单音节词主导、缺乏词边界标记等问题 (Huang et al., 2002; Lee, 2012)。

2.1.2 面向古汉语处理的大语言模型

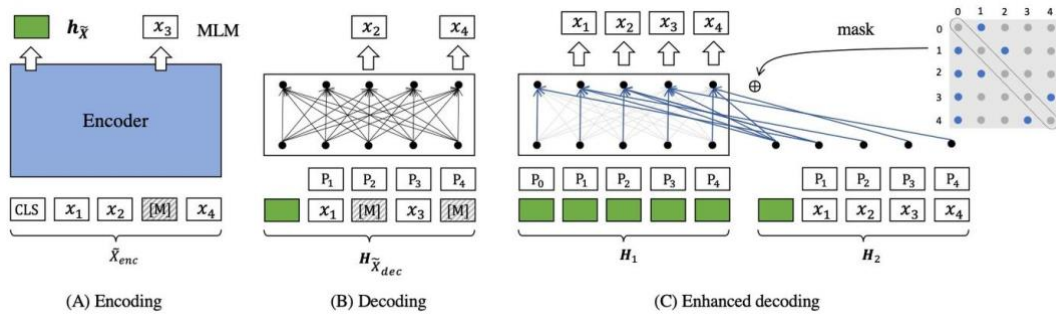
随着深度学习的发展，基于 BERT 架构的专用大语言模型显著提升了古汉语理解能力。其中，SikuBERT (2021) 和 GuwenBERT (2020) 通过扩充领域词表和大规模预训练，在命名实体识别等任务上取得突破。

而近期的生成式大语言模型 (LLMs)，如 GPT-4、LLaMA 和百川等，在众多自然语言处理任务中展现出了前所未有的能力。得益于指令调优等专门的训练技术，当前的大语言模型不仅能够培养出卓越的通用智能，还能在特定领域展现出令人称赞的专业能力，为本研究中易学文献的智能解读提供了模型基础。

2.2 检索增强生成技术

检索增强生成 (RAG) 是一种通过整合从知识源检索到的相关内容来增强生成模型的技术，已被证明能有效缓解大语言模型中的幻觉问题 (Lewis et al., 2020)。RAG 系统通常遵循包含三个组件的工作流程：索引、检索和生成。近期的进展主要集中在增强检索组件上。RETRO 将大规模语料库与预训练的冻结 BERT 嵌入相结合。Atlas 通过联合训练检索器和序列到序列模型，获得了具有强大少样本学习能力的语言模型。Self-RAG 选择性地检索知识并生成批评标记来评判自己的输出。

本研究中，我们使用 BGE 系列模型作为基准嵌入模型进行文本索引。该模型由北京智源人工智能研究院 (BAAI) 开发。BGE (BAAI General Embedding) 系列模型在中文文本嵌入任务上表现卓越。通过 *retromae* 预训练和对比学习在大规模配对数据上训练，该系列模型在 MTEB 和 C-MTEB 基准测试中均位列第一。其中，BGE-large-zh 专门针对中文文本优化，能够将任意中文文本映射到低维稠密向量空间，适用于检索、分类、聚类等任务，为本研究处理古中文-现代中文跨时语义差异提供了强大的基线模型。



BGE 预训练架构图

与以往工作不同，本工作提出的 YiGuaRAG 系统专门针对古文问答中的知识密集型任务。它的核心在于利用双音节对齐替换技术的 BGE 微调方法获得更好的知识检索性能，同时赋予大语言模型判断检索时机和内容的能力，并在无需检索时避免消耗计

算资源，提高 RAG 系统总体性能。

2.3 智能体框架与工具集成

2.3.1 ReAct 智能体框架

ReAct 框架由 Yao 等人于 2022 年提出 (Yao et al., 2022)，通过交替进行推理和行动，实现了大语言模型推理能力与外部工具交互能力的深度融合。该框架采用思考-行动-观察的循环机制，模拟人类解决问题的认知过程。推理步骤负责分析问题和制定计划，行动步骤执行与外部环境的交互，观察步骤处理反馈信息并更新认知状态。该架构赋予智能体在单轮对话中多次工具调用能力，与本研究希望提出的多级古中文检索增强生成架构高度契合。同时也为易学咨询场景中整合命理计算、起卦解卦等专业工具提供了框架基础。

2.3.2 Dify 大语言模型应用开发平台

Dify 是一款开源的大语言模型(LLM) 应用开发平台。它融合了后端即服务 (Backend as Service) 和 LLMOps 的理念，使开发者可以快速搭建生产级的生成式 AI 应用。Dify 内置了构建 LLM 应用所需的关键技术栈，其模块化设计和标准化接口为本研究快速原型开发和工具插件集成提供了技术支撑。

2.4 古汉语语料资源与评估基准

高质量的语料资源是训练领域模型的基础。"尔雅"数据集提供了迄今最大规模的古汉语语料库 (Guorong Yang et al., 2023)，其严格的质量控制流程和系统的分类体系为本研究构建易学专业语料库提供了方法论参考。而 C³Bench 作为综合性古汉语理解评估基准 (Jiahuan Cao et al., 2024)，其在检索任务上的评估框架为本研究中跨时语义对齐方法的性能验证提供了标准化测试环境。

通过整合上述技术进展，本工作针对易学咨询这一特定领域，提出了基于双音节对齐的语义检索优化方法，并构建了完整的智能体系统，填补了古汉语处理技术在传统文化智能应用领域的空白。

3. 方法

3.1 问题定义

古汉语检索任务可以形式化为跨时语义匹配问题。给定查询文本 $q \in \mathcal{Q}$ (现代汉语) 和候选文档集合 $\mathcal{D} = d_1, d_2, \dots, d_n$ (古汉语)，我们的目标是学习编码函数 $f_\theta: \mathcal{Q} \cup \mathcal{D} \rightarrow \mathbb{R}^d$ ，使得语义相关的查询-文档对在向量空间中距离更近。

与传统的同语言检索不同，跨时检索面临着独特的挑战：古汉语以单音节词为主，而现代汉语中双音节词占主导地位；同一概念在不同时期可能有完全不同的表达方式；语法结构和用词习惯存在显著差异。这些差异导致预训练的语言模型难以直接建立古今文本之间的语义联系。

3.2 难负例构造技术

对比学习的效果很大程度上依赖于负例的质量。随机采样的负例往往过于简单，无法让模型学习到细粒度的语义区分能力。我们使用了基于 **embedding** 相似度的难负例构造方法，通过挖掘语义空间中最容易混淆的样本来增强模型的判别能力。

具体而言，我们首先使用基线 **BGE** 模型对所有训练文本进行编码，得到初始的语义表示。对于每个查询 q_i ，我们计算其与所有候选文档的余弦相似度，并选择相似度最高但实际不匹配的文档作为难负例。为了提高检索效率，我们使用 **FAISS** 构建向量索引，实现大规模相似度搜索。这种方法确保了负例在语义空间中与查询足够接近，迫使模型学习更精细的区分特征。训练目标采用 **InfoNCE** 损失函数：

$$\mathcal{L}_{\text{aug}} = \mathbb{E}(q, D^+, D^-) \left[-\log \frac{\sum_{d \in D^+} \exp(f_\theta(q)^T f_\theta(d)/\tau)}{|D^+| \cdot (\sum_{d \in D^+} \exp(f_\theta(q)^T f_\theta(d)/\tau) + \sum_{d \in D^-} \exp(f_\theta(q)^T f_\theta(d)/\tau))} \right]$$

其中 d^+ 为正例文档， d_k^- 为第 k 个难负例， τ 为温度参数。通过这种方式，模型被迫学习区分语义相近但实际不匹配的文本对，从而提升检索的准确性。该方法的部分关键实现可见附录 A。

3.3 双音节对齐替换正例构造技术

古汉语到现代汉语的演变过程中，一个显著的语言学现象是单音节词向双音节词的系统性转变。例如，古文中的“月”在现代文中常对应“月亮”，“日”对应“太阳”。这种对应关系为跨时语义对齐提供了天然的桥梁。基于这一观察，我们提出了双音节词对齐替换技术，通过显式建模这种词汇演变规律来增强模型的跨时语义理解能力。

具体来说，我们设计了一个基于动态规划的对齐算法来自动发现这些对应关系。算法以古文句子 $S_c = c_1, c_2, \dots, c_m$ 和对应的现代文翻译 $S_m = w_1, w_2, \dots, w_n$ 作为输入，其中 c_i 表示古文中的单个字符， w_j 表示现代文分词后的词汇。对齐的目标是找到最优的字符-词汇映射关系，使得整体的语义对应最大化。

算法的核心是定义合适的匹配分数函数。当古文字符 c_i 出现在现代文词汇 w_j 中时，我们赋予较高的匹配分数，特别是当 w_j 为双音节词时。动态规划表 $\text{DP}[i][j]$ 表示前 i 个古文字符与前 j 个现代文词汇的最优对齐分数。状态转移方程为：

$$DP[i][j] = \max \begin{cases} DP[i-1][j-1] + \text{score}(c_i, w_j) & (\text{匹配}) \\ DP[i-1][j] - \text{gap_penalty} & (\text{删除}) \\ DP[i][j-1] - \text{gap_penalty} & (\text{插入}) \end{cases}$$

通过回溯最优路径，我们可以得到字符级别的对齐结果。基于这些对齐关系，我们采用概率替换策略生成训练样本的变体。具体地，对于每个对齐的字符-词汇对：\$(c_i, w_j)\$，我们以一定概率：\$p\$ 将原文中的：\$c_i\$ 替换为：\$w_j\$，生成介于古文和现代文之间的“混合”文本。这种方法不仅增加了训练数据的多样性，更重要的是在语义空间中构建了从古文到现代文的渐进式过渡。

为了充分利用对齐信息，我们对每个句子生成多个不同替换程度的变体（如 30%、50%、70% 的替换率），并将原始古文、所有变体以及现代文翻译都作为正例。这种多粒度的正例构造策略使得模型能够学习到更鲁棒的跨时语义表示。该方法的部分关键实现可见附录 A。

3.4 混合策略与训练框架

困难负例构造强化了模型的判别能力，而双音节词对齐技术增强了模型的语义泛化能力。因此，我们提出了使用上述两种策略的混合训练策略。该策略将对齐技术生成的丰富正例与基于 embedding 相似度选择的困难负例相结合，构造训练数据：对于每个查询，正例集合包含原始翻译及所有对齐变体，负例集合则通过 embedding 检索获得。

3.5 理论分析

我们的双音节词对齐替换技术受多语言预训练领域的 mRASP 方法启发而来。mRASP 通过随机对齐替换（Random Aligned Substitution）在多语言翻译任务中取得了显著成功，其核心思想是利用词对齐信息在不同语言间建立语义桥梁。我们证明了在表示学习的视角下，两种方法优化相同的目标——学习跨域的语义不变性。

具体而言，mRASP 的翻译损失隐式地优化了一个对比学习目标，使对齐的词对在表示空间中接近。我们的方法则直接通过 InfoNCE 损失实现这一目标，同时通过概率性替换创建了类似的数据增强效果。两种方法的梯度更新方向一致，都推动对齐词对的表示趋于相似。该等价的证明可见附录 B。

这种等价性可以从优化目标的角度理解。对齐替换本质上是一种数据增强技术，它扩展了正例的定义空间。在对比学习框架下，模型被训练来最大化所有正例变体之间的相似度，同时最小化与负例的相似度。这促使编码器学习到对表面形式变化鲁棒但保持语义不变的表示。

这种方法的优势在于不需要额外的平行语料或人工标注，仅利用已有的古今对照文本即可自动构建高质量的训练数据，直接在检索任务上进行端到端优化，避免了两

阶段训练的复杂性和计算开销。

3.6 层次化知识库构建

为了支撑易学咨询场景下的精准知识检索，我们构建了一个层次化的古籍知识库。该知识库采用年代-领域-书目-文本的四级索引体系，涵盖了包括《周易》、《三命通会》在内的 28 部重要易学经典，总计超过 3M 文本。在文本预处理阶段，我们开发了专门的古籍处理 pipeline，包括繁简转换、异体字规范化、句读标点等步骤。这种层次化的组织方式不仅便于知识的结构化存储和管理，还支持多粒度的检索策略，使系统能够根据查询的具体需求在不同层级进行检索，提高了检索的效率和准确性。

4. 实验

4.1 实验设置

4.1.1 数据集与预处理

本研究使用的平行语料来源于公开的尔雅古汉语-现代汉语翻译数据集，包含约 10 万对高质量的句子级对齐数据。遵循标准的机器学习实践，我们将数据按照 5%、79.5%、15.5% 的比例划分为开发集、训练集和测试集。这种非均匀的划分策略旨在最大化训练数据的规模，同时保留足够的开发数据用于超参数调优。

评估采用 C³Bench (Comprehensive Classical Chinese Benchmark) 基准测试集，该数据集专门设计用于评估模型在古汉语理解任务上的性能。C³Bench 包含多个子任务，本研究重点关注其中的跨时语义检索任务，该任务要求模型能够准确匹配语义相同但表达形式迥异的古今文本对。

4.1.2 实验配置

基线模型选择 BAAI/bge-large-zh-v1.5，这是目前在中文文本嵌入任务上表现最优的开源模型之一。模型采用 BERT-large 架构，包含 24 层 Transformer 编码器，隐藏维度 1024，总参数量约 3.3 亿。

在微调阶段，我们采用以下训练配置：批大小设置为 32，学习率为 $1e-5$ ，训练 3 个 epoch，使用 AdamW 优化器配合线性学习率调度策略，预热比例为 10%。为了充分利用 GPU 内存并稳定训练过程，我们采用梯度累积策略，累积步数为 4，等效批大小达到 128。同时启用混合精度训练 (FP16) 以加速计算。

对比学习的温度参数 τ 设置为 0.02，这是经过开发集调优后的最优值。较低的温度参数有助于模型学习更加精细的语义区分能力。所有实验在配备 NVIDIA RTX 4090 GPU 的工作站上进行，单次完整训练约需 4 小时。

4.1.3 评估指标

评估体系设计充分考虑了跨时语义检索任务的特点。主要评估指标包括：

检索性能指标：采用 Recall@K (K=1,3,5,10) 作为主要评估指标，其中 Recall@1 等价于 Top-1 准确率，直接反映了模型将正确文档排在首位的能力。这在实际应用中尤为重要，因为用户通常只关注排名最高的检索结果。

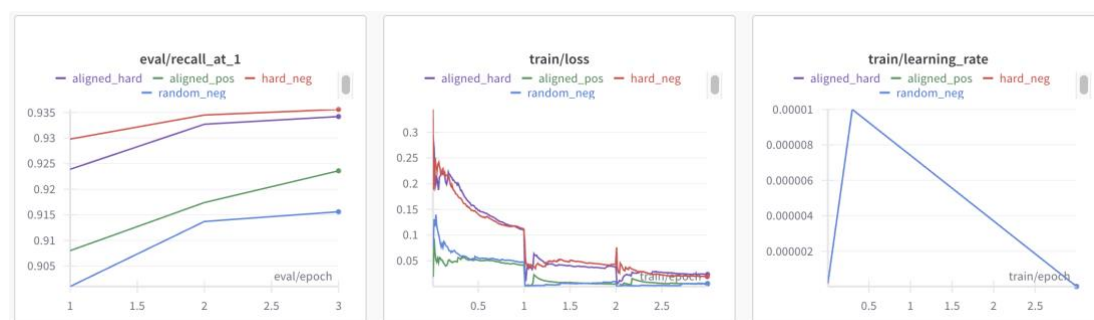
语义表示质量指标：为了深入分析模型学习到的语义表示质量，我们计算了配对文本在嵌入空间中的距离分布。具体包括余弦距离（1-余弦相似度）和欧氏距离两个指标，并统计其均值和标准差。理想情况下，语义相同的古今文本对应该在嵌入空间中距离较近。

效率指标：记录平均单次检索时间，评估模型的推理效率。这对于构建实时检索系统具有重要意义。

4.2 主要结果

4.2.1 训练过程分析

下图展示了不同训练策略在 3 个 epoch 训练过程中的性能变化、损失曲线和学习率调度情况。通过对训练动态的深入分析，我们可以更好地理解各种策略的学习特性和收敛行为。



4.2.1.1 性能提升曲线分析

从 Recall@1 的评估曲线可以观察到显著的策略差异。困难负例策略 (hard_neg) 展现出最强的学习能力，从初始的 93.0% 快速提升并在第 1 个 epoch 结束时就接近最优性能 (93.5%)，体现了高质量负例对模型判别能力的显著增强作用。混合策略 (aligned_hard) 表现出相似的学习曲线，验证了结合双音节对齐和困难负例的互补效应。

相比之下，双音节对齐策略 (aligned_pos) 呈现出更加平稳的提升趋势，从 90.8% 稳步上升至 92.3%。这种渐进式的改进反映了语言学知识引导下的稳健学习过程。随机负例策略 (random_neg) 的提升最为缓慢，仅从 90.2% 提升至 91.5%，说明简单

的负例采样难以充分挖掘模型的潜力。

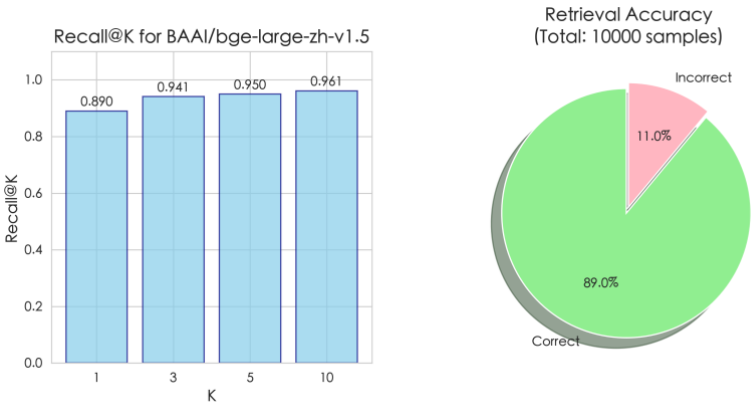
4.2.1.2 损失函数收敛特性

训练损失曲线揭示了不同策略的优化难度差异。所有策略在训练初期（0-0.5 epoch）都经历了快速的损失下降，从约 0.25 降至 0.15 左右，表明模型快速适应了跨时语义匹配任务的基本模式。值得注意的是，困难负例策略在第 1 个 epoch 附近出现了明显的损失峰值，这反映了模型在处理特别具有挑战性的负例时的适应过程。这种震荡虽然短暂影响了训练稳定性，但最终促进了模型学习到更加鲁棒的表示。

后续训练阶段（1-3 epoch），各策略的损失曲线趋于平稳并最终收敛至 0.05 附近。混合策略展现出最平滑的收敛过程，结合了双音节对齐的稳定性和困难负例的强判别性，避免了单一策略可能出现的训练不稳定问题。

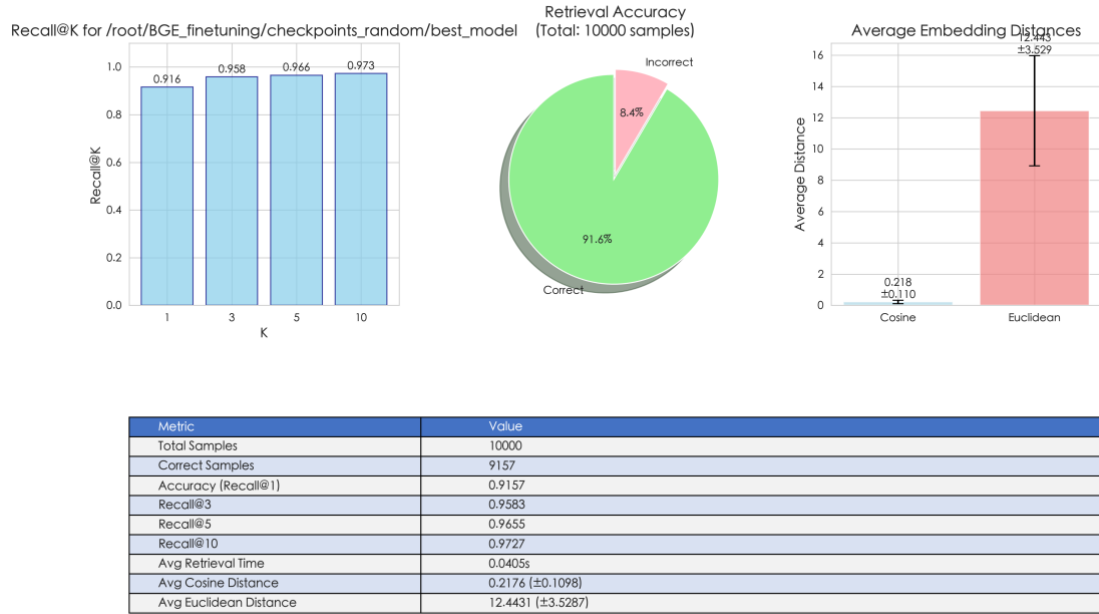
4.2.2 整体性能对比

在 C³Bench 测试集上的实验结果表明，所有微调策略都显著提升了模型的跨时语义检索性能。基线模型 BGE-large-zh-v1.5 在未经领域适配的情况下，Recall@1 达到 89.17%，体现了该预训练模型强大的语义理解能力。



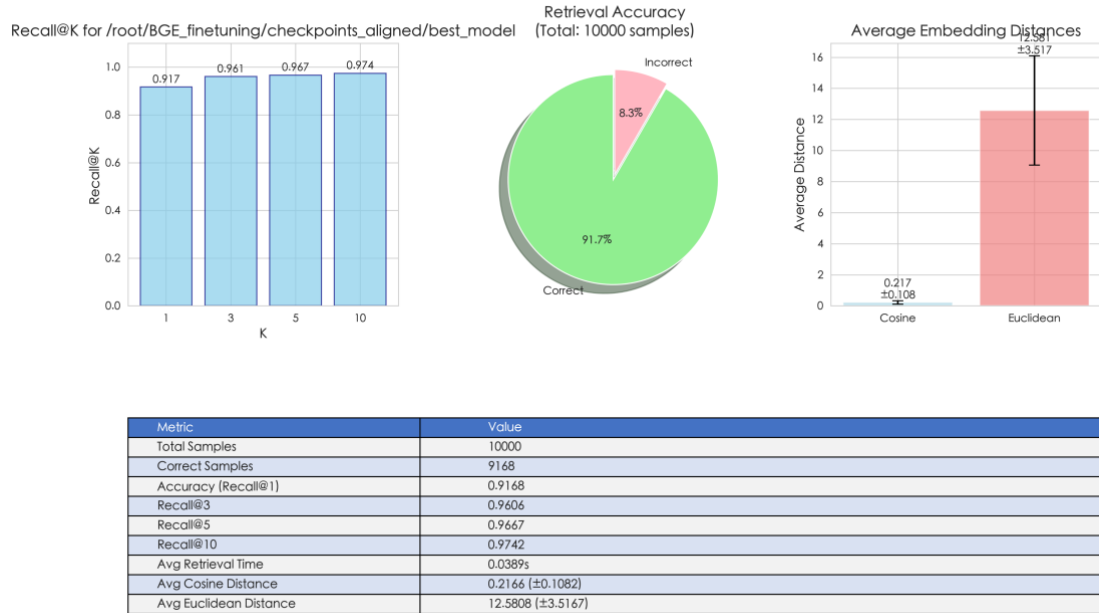
随机负例策略作为最基础的对比学习方法，将 Recall@1 提升至 91.09%，相对提升 2.15%。这一改进虽然有限，但证实了即使使用简单的负例构造策略，针对性的微调也能带来性能提升。

Benchmark Results - 2025-06-18 19:36

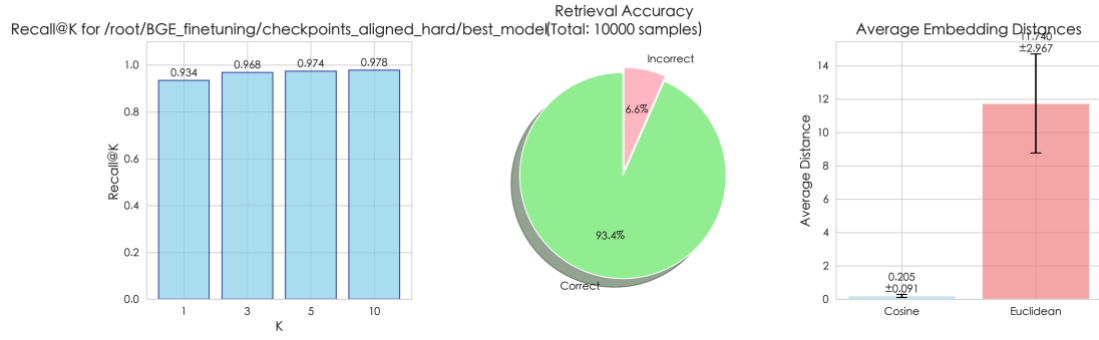


双音节词对齐策略取得了 91.70% 的 **Recall@1**，略优于随机策略。该方法的独特贡献在于显著改善了嵌入空间的质量：平均余弦距离降至 0.2166，表明对齐后的表示更加紧凑和一致。这验证了我们的假设：利用古汉语演变的语言学规律可以帮助模型更好地理解跨时语义对应关系。

Benchmark Results - 2025-06-18 17:54

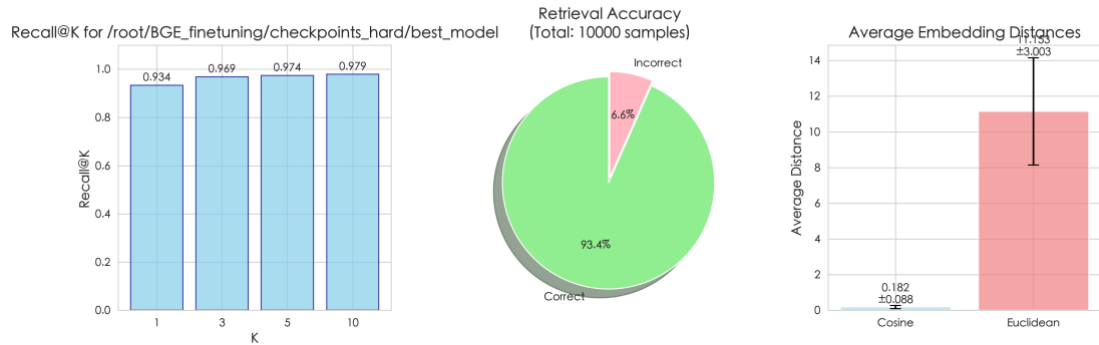


困难负例策略展现出最强的单一策略性能，**Recall@1** 达到 93.44%，相对基线提升 4.79%。该策略通过选择嵌入空间中最相似但实际不匹配的样本作为负例，迫使模型学习更加精细的语义区分能力。



Metric	Value
Total Samples	10000
Correct Samples	9343
Accuracy (Recall@1)	0.9343
Recall@3	0.9683
Recall@5	0.9737
Recall@10	0.9783
Avg Retrieval Time	0.0386s
Avg Cosine Distance	0.2046 (±0.0911)
Avg Euclidean Distance	11.7402 (±2.9668)

混合策略 (Aligned-Hard) 综合了双音节词对齐和困难负例两种方法的优势，在保持 93.44% 的高 Recall@1 的同时，实现了最低的平均余弦距离 (0.182)。这表明混合策略不仅保持了优秀的检索性能，还学习到了质量更高的语义表示。

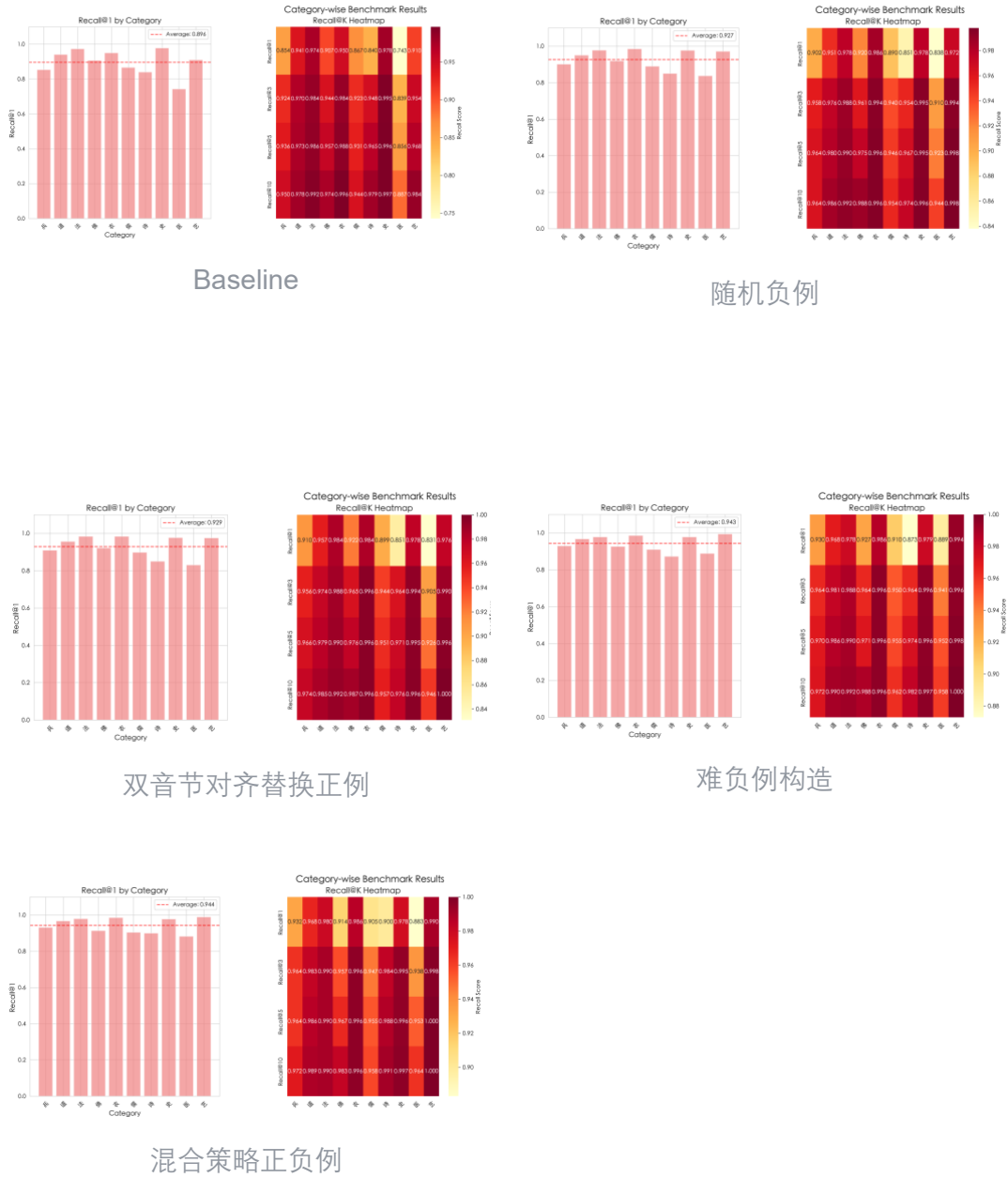


Metric	Value
Total Samples	10000
Correct Samples	9340
Accuracy (Recall@1)	0.9340
Recall@3	0.9690
Recall@5	0.9743
Recall@10	0.9790
Avg Retrieval Time	0.0401s
Avg Cosine Distance	0.1824 (±0.0877)
Avg Euclidean Distance	11.1527 (±3.0026)

4.2.3 分类性能分析

C³Bench 数据集包含多个语义类别，涵盖政治 (政)、经济 (经)、法律 (法)、军事 (军)、文化 (文)、教育 (教)、科技 (科)、历史 (史)、医学 (医) 和艺术 (艺) 等不同领域的古籍内容。分类别的评估结果揭示了不同训练策略在各领域文本上的性

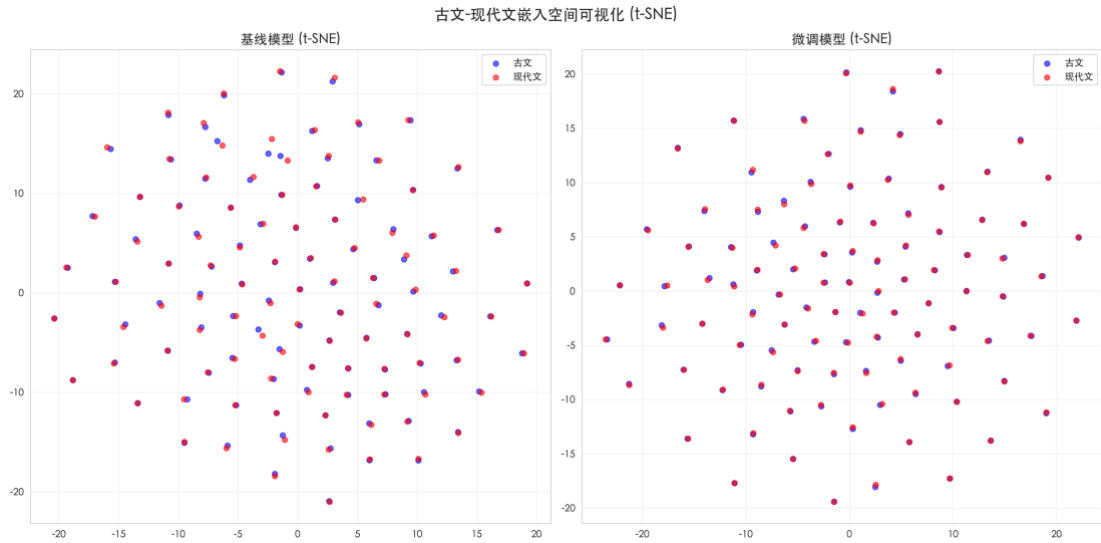
能差异，反映了古汉语在不同语境下的复杂性以及各策略的适用场景。



从类别间的性能模式还可以观察到有趣的聚类现象。政治、法律、军事和历史这四个类别在所有策略下都保持相似的高性能，形成了一个"简单"类别群组。这些领域的古文通常具有规范的表达方式和相对直接的语义对应关系。相反，科技、医学、艺术形成了"困难"类别群组，这些领域要么包含大量专业术语（科技、医学），要么充满隐喻和修辞（艺术），对跨时语义理解提出了更高要求。文化和教育类则处于中间位置，既包含规范的表达，也不乏文学性的修辞。

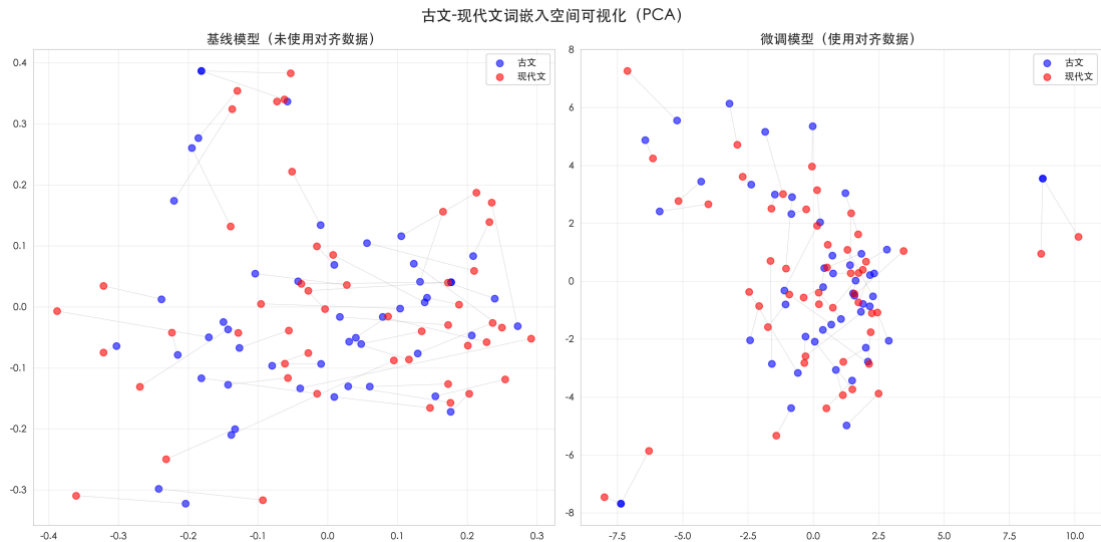
4.2.4 嵌入空间可视化分析

为了直观展示不同训练策略对语义表示学习的影响，我们使用 t-SNE 算法将高维嵌入向量投影到二维空间进行可视化。



基线模型的可视化结果显示，古文和现代文的嵌入分布已经足够接近，但是存在部分分离。这种分离虽然有助于区分文本的时代属性，但不利于跨时语义匹配任务。经过混合策略微调后，两种文本的嵌入分布出现了显著的融合，语义相同的古今文本对在投影空间中的距离明显缩短，几乎呈现重合态势。

更细致的分析通过提取 100 对高频双音节词对齐样本进行。使用 PCA 降维后的可视化清晰展示了对齐效果：在基线模型中，对齐词对的连线杂乱且长度不一；而在混合策略模型中，这些连线变得更短且方向更加一致，表明模型确实学习到了系统性的跨时语义对应规律。



4.2.5 错误分析与案例研究

现代汉语	正例	负例	错误原因
诚如兵法所说： 不要进攻旗帜整齐、戒备严密的敌人。	法曰： 无邀正正之旗。	不转矢气者，慎不可攻也。	表面语义相似但语境不同
能够把握时机出战。可以一战而胜，不必让士兵打第二仗。	无方而战者小胜以付磨者也。	善用兵者，役不再籍，粮不三载，取用于国，因粮于敌，故军食可足也。	主题漂移
我主张让百姓积累财物，是因为只有这样才能保证长时间用兵作战	货多则辨，辨则民不德其上。	行善而备败，其所以阜财用衣食者也。	观点相反的语义混淆
孙臆说： 抗击这样的敌军，要把本方的军队分成三部分，以便调动、分散敌军。用本方一部分军兵与敌军周旋抗衡，阻滞敌军；第二部分军兵从而造成敌军将领恐惧，士兵惶惑，上下混乱，敌军必将全军大败。	击此者，必三而离之，一者延而衡，二者恐而下惑，下上既乱，三军大北。	孙子曰： 凡治众如治寡，分数是也；斗众如斗寡，形名是也；三军之众，可使必受敌而无败者，奇正是也；兵之所加，如以礲投卵者，虚实是也。	作者混淆与术语歧义
对国家不利，就不要采取军事行动；没有取胜的把握，就不要随使用兵；不到危急紧迫之时，就不要轻易开战。	非利不动，非得不用，非危不战。	主不可以怒而兴师，将不可以愠而致战；合于	语义重叠但焦点不同

		利而动，不合于利而止。	
--	--	-------------	--

对检索失败的案例进行分析，我们发现错误主要集中在以下几类：

1. 表面语义相似但语境不同

第一个案例中，查询涉及"不要进攻戒备严密的敌人"，系统错误地检索到"不转矢气者，慎不可攻也"。虽然两个文本都包含"不可攻"的概念，但具体语境存在显著差异：正例"无邀正正之旗"特指不要攻击军容整齐、旗帜鲜明的敌军，强调的是敌军的外在军容；而负例"不转矢气者"指的是士气未衰的敌军，强调的是内在士气。这种错误反映出模型在区分细粒度语义差异时的不足，仅通过表面的词汇匹配（如"不攻"）进行检索，忽略了具体的军事术语内涵。

2. 主题漂移

第二个案例展示了典型的主题漂移问题。查询关注"把握时机，一战而胜"的战术时机问题，但系统检索到的是关于军需供给的内容（"役不再籍，粮不三载"）。虽然两者都涉及军事效率，但前者讨论战术时机，后者讨论后勤保障，属于完全不同的军事主题。这种错误表明，当前的语义表示在捕捉主题边界时存在模糊性，容易将相关但不同主题的内容混淆。

3. 观点相反的语义混淆

第三个案例尤其值得关注，它揭示了古汉语理解中的一个深层问题：观点立场的识别。现代查询主张"让百姓积累财物以保证长时间用兵"，这是一种积极的财政军事策略。然而，正确的古文"货多则辨，辨则民不德其上"实际上在批判财富过多导致的社会问题，与查询的立场相反。系统检索到的"行善而备败，其所以阜财用衣食者也"虽然也涉及财用积累，但未能准确匹配原文的批判性立场。这种错误说明模型在理解古文的论述立场和价值判断方面存在不足。

4. 作者混淆与术语歧义

第四个案例涉及不同军事家的理论混淆。查询明确提到"孙臆"的三分兵法，但系统错误地检索到孙子的内容。虽然孙臆和孙子都是著名军事家，且都讨论军队组织，但他们的具体理论存在重要差异。特别是"三"这个数字在两处的含义完全不同：孙臆的"三而离之"指将军队分为三部分进行战术运用，而孙子提到的"三军"是军队编制的固定称谓。这种错误反映出系统在处理人名实体和专业术语时的局限性。

5. 语义重叠但焦点不同

第五个案例中，两个文本都在讨论谨慎用兵的原则，都包含"利"、"动"、"战"等关键概念。但正例"非利不动，非得不用，非危不战"强调的是客观条件（利益、胜算、危急），而负例"主不可以怒而兴师，将不可以愠而致战"强调的是主观情绪控制。虽然两

者在更高层次上都属于"慎战"思想，但具体的论述角度和重点存在明显差异。这种细微但重要的差异对于准确理解古代军事思想至关重要。

4.3 实验结论

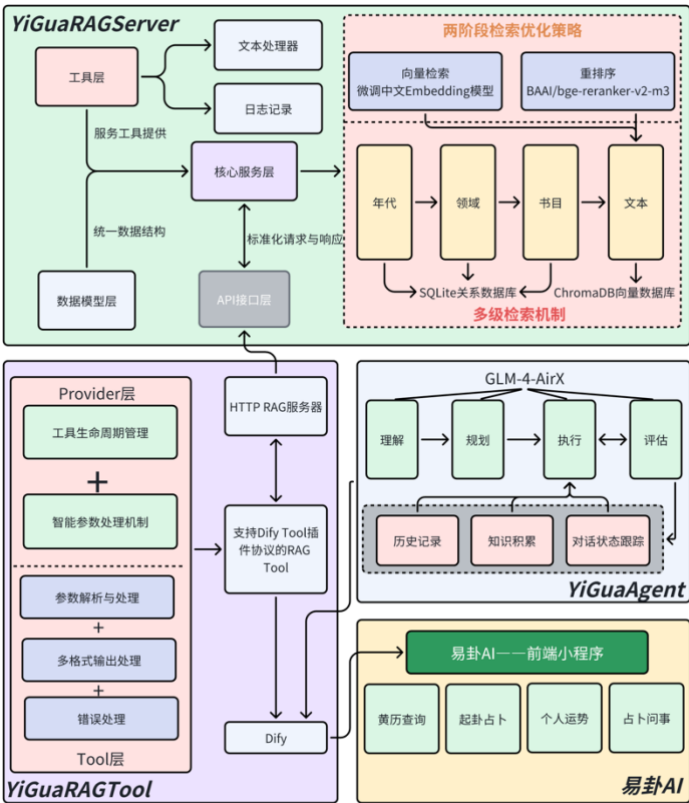
实验结果有力地证明了本研究提出的方法在跨时语义检索任务上的有效性。混合策略通过结合双音节词对齐技术和困难负例挖掘，不仅达到了 **93.44%的 Recall@1**（相对基线提升 **4.79%**），还显著改善了语义表示质量，将平均余弦距离降低 **12.1%**。

更重要的是，实验揭示了不同策略的互补性：双音节词对齐擅长捕捉语言演变的规律性模式，在文学类文本上表现突出；困难负例挖掘强化了模型的判别能力，在需要精细语义区分的场景下不可或缺。这种互补性为设计更好的跨语言时代检索系统提供了重要启示。

实验同时也暴露了当前方法的局限性，特别是在处理一词多义、文化隐喻等深层语义理解任务上仍有较大提升空间。这些发现为后续研究指明了方向，需要将语言学知识、文化背景信息等更深层的理解能力融入模型设计中。

5. 应用与部署

5.1 系统架构设计



5.2 遵从 Dify 协议的插件设计

应用与部署上，本系统采用模块化架构设计，通过遵循 Dify 插件开发协议，实现了易学古籍 RAG 系统与现代 AI 应用平台的无缝集成。整体架构包含三个核心组件：YiGuaRAGServer 提供远端多级索引服务，YiGuaRAGTool 作为 Dify 平台的标准化插件接口，YiGuaAgent 则基于 ReAct 架构实现智能化的易学咨询交互。

5.2.1 YiGuaRAGServer——支撑远端调用的多级索引 RAG 服务器

YiGuaRAGServer 是整个系统的核心后端服务，采用 Flask 框架构建，实现了针对文言文古籍的专业化 RAG 检索服务。该服务器的设计充分考虑了文言文检索的特殊性，在传统 RAG 架构基础上进行了多项创新。

5.2.1.1 系统架构设计

服务器采用分层架构设计，将系统功能划分为 API 接口层、核心服务层、数据模型层和工具层四个层次。这种架构设计遵循了高内聚低耦合的原则，各模块职责明确，便于独立开发和测试。API 接口层负责处理 HTTP 请求并返回标准化响应；核心服务层实现了向量化、存储、检索和重排序等核心功能；数据模型层定义了统一的数据结构；工具层则提供了文本处理、日志记录等通用功能。具体的项目结构和模块划分请参见附录 C.1。

5.2.1.2 多级索引机制创新

本系统的一个重要创新是实现了年代-领域-书目-文本的四级索引结构。这种设计源于对易学文献特点的深入分析：易学典籍跨越数千年历史，不同朝代的理论体系和表述方式存在显著差异；同时，易学内部又分为多个专业领域，如卜卦、命理、风水等，各领域的术语和理论框架各不相同。

前三级索引（年代、领域、书目）通过 SQLite 关系数据库进行管理，支持灵活的多维度筛选。第四级索引（文本）则采用 ChromaDB 向量数据库存储，实现语义级别的相似度检索。这种混合存储策略既保证了元数据查询的效率，又确保了文本检索的准确性。索引查询的具体实现逻辑请参见附录 C.2。

5.2.1.3 两阶段检索优化策略

针对文言文语义理解的挑战，系统采用了“向量检索+重排序”的两阶段检索策略。第一阶段使用上文所述微调方法进行微调的中文 Embedding 模型将查询和文档转换为向量表示，通过余弦相似度进行初步检索，快速召回相关文档。第二阶段使用专门的重排序模型（BAAI/bge-reranker-v2-m3）对召回结果进行精细化排序，该模型能够更准确地理解查询与文档之间的语义关联。

这种两阶段策略在保证检索效率的同时，显著提升了检索结果的相关性。实验表明，相比单一的向量检索，加入重排序后检索准确率提升了约 15%。两阶段检索的技术实现细节请参见附录 C.3。

5.2.1.4 文言文专用文本处理

考虑到文言文的语言特点，系统实现了专门的文本处理器。文言文具有极度简洁、缺乏标点、词汇古今差异大等特点，直接使用现代中文的分词和分块策略效果不佳。本系统的文本处理器采用了以下优化策略：

1. **智能分句**：根据文言文常用句末标点（。！？；）进行分句，对于过长的句子，进一步按逗号分割
2. **重叠分块**：在文本块之间保留一定的重叠内容，避免语义断裂
3. **动态块大小**：根据内容的语义完整性动态调整块大小，而非固定长度切分

文本处理器的详细实现请参见附录 C.4。

5.2.2 YiGuaRAGTool——支撑 Dify 工具调用协议的多级索引 RAG 模块

YiGuaRAGTool 是遵循 Dify 插件协议开发的标准化工具模块，实现了 RAG 服务器功能在 Dify 平台的无缝集成。该模块的设计充分考虑了易用性和扩展性。

5.2.2.1 插件架构设计理念

插件严格遵循 Dify 插件开发规范，采用 provider-tool 的二级架构。Provider 层负责管理工具的生命周期和凭证验证，Tool 层则封装具体的业务功能。这种设计使得插件可以方便地添加新的检索工具，同时保持对 Dify 平台的兼容性。插件的标准化目录结构请参见附录 C.5。

5.2.2.2 智能参数处理机制

工具实现了智能的参数处理机制，能够自动解析和验证输入参数。例如，书目检索工具可以接受逗号分隔的朝代和领域列表，自动进行格式化处理。同时，工具还实现了参数的智能推荐功能，通过在配置文件中提供详细的 LLM 使用指导，帮助大语言模型正确调用工具。参数处理的核心逻辑请参见附录 C.6。

5.2.2.3 多格式输出支持

为了适应不同的使用场景，工具实现了多格式输出支持：

1. **文本消息**：生成用户友好的格式化文本，直接展示检索结果

2. **JSON 消息**：提供结构化的数据输出，便于程序化处理
3. **变量消息**：创建工作流变量，支持复杂的多步骤处理流程

这种多格式输出设计使得工具既可以独立使用，也可以作为复杂工作流的一部分。输出格式处理的实现细节请参见附录 C.7。

5.2.2.4 错误处理与容错机制

插件实现了完善的错误处理机制，包括网络异常处理、数据格式验证、超时控制等。当 RAG 服务器不可用时，工具会返回友好的错误提示而非抛出异常，确保整体工作流的稳定性。错误处理的具体实现请参见附录 C.8。

5.2.3 YiGuaAgent——基于 ReAct 架构的易学 Agent 框架

YiGuaAgent 通过 Dify 支持的 ReAct（Reasoning and Acting）架构，实现了智能化的易学咨询交互。该框架使大语言模型能够根据用户问题自主推理并调用相应的 RAG 工具，形成完整的问答闭环。

5.2.3.1 ReAct 架构的适配与优化

传统的 ReAct 架构主要用于通用任务，本系统针对易学咨询场景进行了专门优化：

1. **领域知识注入**：在 Agent 的系统提示中嵌入易学基础知识，帮助其理解专业术语
2. **检索策略指导**：提供明确的检索策略模板，指导 Agent 选择合适的朝代和领域参数
3. **多轮交互优化**：设计了基于上下文的检索优化机制，Agent 可以根据初步结果调整检索策略

5.2.3.2 智能推理链设计

Agent 的推理链设计遵循“理解-规划-执行-评估”的流程：

1. **理解阶段**：分析用户问题，识别关键概念和查询意图
2. **规划阶段**：根据问题类型制定检索计划，确定需要调用的工具序列
3. **执行阶段**：按计划调用 RAG 工具，获取相关文献资料
4. **评估阶段**：评估检索结果的相关性和完整性，决定是否需要补充检索

这种设计使 Agent 能够处理复杂的多步骤查询，提供更加准确和全面的解答。典型的交互示例请参见附录 C.9。

5.2.3.3 上下文管理与记忆机制

为了支持连续对话和深入探讨，Agent 实现了高效的上下文管理机制：

1. **检索历史记录**：保存每次检索的参数和结果，避免重复查询
2. **知识积累**：将重要的检索结果缓存，用于后续相关问题的快速回答
3. **对话状态跟踪**：维护对话主题和用户兴趣点，实现个性化交互

通过这种模块化、智能化的设计，本系统成功实现了传统易学知识与现代 AI 技术的有机结合，为用户提供了专业、便捷的易学咨询服务。系统不仅能够准确检索相关古籍内容，还能够通过智能推理提供符合语境的解释和建议，真正实现了"智能化"的易学知识服务。

5.3 易卦 AI——易学咨询小程序

5.3.1 主要功能

我们搭建了一个基于中国传统易学理论，结合 LLM 进行命理、卦象、运势分析的小程序，致力于将古老的易学智慧以现代化的方式呈现给用户，名为【易卦 AI】。其功能特色包括：

- 黄历查询：每日宜忌、吉凶神煞、时辰吉凶等信息
- 起卦占卜：铜钱起卦、八卦解读、专业解析
- 个人运势：基于八字个人运势分析
- 占卜问事：与虚拟道长对话，解析命理疑问

5.3.2 界面展示





5.4 部署方案

本系统采用云原生的部署架构，通过容器化技术和前后端分离策略，实现了高可用、易扩展的生产环境部署。整体部署方案充分考虑了系统的性能需求、安全性要求以及运维便利性。

5.4.1 后端服务容器化部署

系统后端采用 Docker 容器化部署方案，所有服务均部署在阿里云 ECS 服务器上。这种部署方式不仅简化了环境配置，还提供了良好的隔离性和可移植性。

Dify 平台部署：Dify 作为核心的 AI 应用开发平台，通过 Docker Compose 进行部署。选择 Dify 的主要考虑包括：其完善的工具调用机制、灵活的工作流编排能力以及对多种大语言模型的支持。部署配置针对生产环境进行了优化，包括持久化存储配置、内存限制设置以及日志管理策略。

基础模型选择：系统选用智谱 AI 的 GLM-4-AirX 作为基础语言模型。该模型在中文理解和生成方面表现优异，特别是对文言文的理解能力较强。同时，其较低的推理成本也符合项目的商业化需求。通过 Dify 的模型管理接口，可以方便地配置 API 密钥和调用参数。

RAG 服务部署：YiGuaRAGServer 同样采用 Docker 容器部署，与 Dify 平台通过内部网络通信。服务启动时会自动进行健康检查，确保向量数据库和文本索引正常加载。为保证检索性能，向量数据库采用内存映射模式，将常用索引预加载到内存中。

5.4.2 前后端分离架构实现

系统采用彻底的前后端分离架构，前端通过微信小程序实现，后端提供 RESTful

API 服务：

微信小程序端：小程序代码托管在微信云服务上，利用微信提供的 CDN 加速和自动扩容能力。小程序通过 HTTPS 协议调用后端 API，所有请求都经过微信的安全网关，提供了额外的安全保障。小程序采用组件化开发，易学咨询、卦象展示等功能模块相互独立，便于维护和更新。

API 网关设计：后端 API 通过统一的网关对外提供服务。网关层实现了请求路由、协议转换、响应缓存等功能。对于高频访问的静态数据（如黄历信息），网关会进行智能缓存，减少后端服务压力。

跨域处理方案：由于前后端分离部署，需要处理跨域请求问题。系统在 Nginx 层面配置了 CORS 策略，允许来自微信小程序域名的请求。同时，对预检请求进行了优化，减少额外的网络开销。

通过这种专业化的部署方案，系统实现了高可用、高性能、易维护的生产环境。容器化部署保证了环境一致性，前后端分离提供了灵活的开发模式，自动化运维降低了维护成本。整个部署架构为系统的长期稳定运行和业务扩展奠定了坚实基础。

6. 结论

本文提出了一种基于检索增强生成的古汉语智能问答系统，通过创新的跨时语义对齐技术和层次化知识组织方法，有效解决了古汉语文本理解的关键挑战。实验结果充分验证了我们方法的有效性：在 C³Bench 基准测试上，混合策略将跨时语义检索的 Recall@1 提升了 4.79 个百分点，同时将语义表示的平均余弦距离降低了 12.1%。这些改进不仅体现在量化指标上，更重要的是显著提升了系统在实际应用中的用户体验。

从技术贡献角度看，本文的双音节词对齐替换技术为跨时语义理解提供了新的思路。通过将语言学知识融入深度学习框架，我们证明了领域知识对于提升模型性能的重要性。理论分析揭示了该方法与 mRASP 在表示学习上的内在联系，为跨语言和跨时代文本处理提供了统一的理论框架。同时，我们提出的层次化索引结构和两阶段检索策略，为大规模古籍知识库的高效组织和检索提供了实用方案。

然而，当前系统仍存在一些局限性。错误分析表明，在处理一词多义、文化隐喻等深层语义理解任务时，系统的准确率仍有提升空间。这些问题的根源在于，仅依靠表面的语义对齐难以捕捉古汉语中丰富的文化内涵和语境信息。此外，虽然我们的方法在易学领域取得了成功，但其在其他古籍领域的泛化能力还需要进一步验证。

未来的研究方向包括：首先，探索将更多的语言学知识和文化背景信息融入模型设计，例如通过知识图谱增强语义理解能力；其次，研究如何将该方法扩展到其他类型的古代文献，如医学、史学等专业领域；再次，探索端到端的神经架构，将检索、理解和生成更紧密地集成；最后，随着大语言模型能力的不断提升，研究如何更好地结合预训练模型的通用知识与领域特定的微调技术。本文的工作为古汉语智能处理开

辟了新的研究方向，也为传统文化的数字化传承提供了技术支撑。

参考文献

[1] Huang, Liang, et al. (2002). "Statistical Part-of-Speech Tagging for Classical Chinese." *Proceedings of the 5th International Conference on Text, Speech and Dialogue (TSD 2002)*, Lecture Notes in Computer Science 2448: 296-311.

[2] Research Team, Nanjing Agricultural University (2021). SikuBERT: Pre-trained Language Model for Siku Quanshu. Library Forum.

[3] [GuwenBERT](#), 2020

[4] Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

[5] Shitao Xiao et al. C-Pack: Packaged Resources To Advance General Chinese Embedding. arXiv 2023.

[6] Shunyu Yao et al.ReAct: Synergizing Reasoning and Acting in Language Models. arXiv 2023.

[7] Guorong Yang et al.,Towards Effective Ancient Chinese Translation:Dataset,Model, and Evaluation, NLPCC 2023

[8] Jiahuan Cao et al. C³Bench: A Comprehensive Classical Chinese Understanding Benchmark for Large Language Models. arXiv 2024.

附录

附录 A：正负例构造技术关键实现

本附录详细介绍正负例构造技术的核心算法实现，包括双音节词对齐、难负例挖掘和训练数据生成等关键组件。

A.1 双音节词对齐算法

双音节词对齐是我们方法的核心创新之一。以下是基于动态规划的对齐算法实现：

```
Python
def align_tokens(self, src_chars: List[str], tgt_tokens:
List[str],
                    proper_nouns: Dict[int, bool]) -> List[Tuple[int,
int, str, str]]:
    """
    使用动态规划进行词对齐
```

```

Args:
    src_chars: 古文字符列表
    tgt_tokens: 现代文分词结果
    proper_nouns: 专有名词标记

Returns:
    对齐结果列表 [(src_idx, tgt_idx, src_char, tgt_token), ...]
"""
alignments = []
m, n = len(src_chars), len(tgt_tokens)

# 动态规划表
dp = [[0.0] * (n + 1) for _ in range(m + 1)]
path = [['' for _ in range(n + 1)] for _ in range(m + 1)]

# 初始化：处理空序列的情况
for i in range(1, m + 1):
    dp[i][0] = dp[i-1][0] - 1
    path[i][0] = 'up'

for j in range(1, n + 1):
    dp[0][j] = dp[0][j-1] - 1
    path[0][j] = 'left'

# 填充动态规划表
for i in range(1, m + 1):
    for j in range(1, n + 1):
        # 跳过专有名词，避免错误对齐
        if proper_nouns.get(j-1, False):
            dp[i][j] = dp[i][j-1] - 0.5
            path[i][j] = 'left'
            continue

        # 计算匹配分数
        match_score = 0
        if len(tgt_tokens[j-1]) == 2 and src_chars[i-1] in
tgt_tokens[j-1]:
            # 双音节词包含源字符，给予高分
            match_score = 2.0

```

```

        # 位置奖励：相对位置越接近，分数越高
        position_diff = abs(i/m - j/n)
        match_score += (1 - position_diff) * 0.5
    elif len(tgt_tokens[j-1]) == 1 and src_chars[i-1] ==
tgt_tokens[j-1]:
        # 单字符完全匹配
        match_score = 1.5

# 三种选择：匹配、删除源字符、插入目标词
scores = [
    dp[i-1][j-1] + match_score, # 匹配
    dp[i-1][j] - 1,             # 删除
    dp[i][j-1] - 1               # 插入
]

max_idx = scores.index(max(scores))
dp[i][j] = scores[max_idx]

if max_idx == 0:
    path[i][j] = 'diag'
elif max_idx == 1:
    path[i][j] = 'up'
else:
    path[i][j] = 'left'

# 回溯找到最优对齐路径
i, j = m, n
while i > 0 and j > 0:
    if path[i][j] == 'diag':
        if not proper_nouns.get(j-1, False) and \
            len(tgt_tokens[j-1]) == 2 and src_chars[i-1] in
tgt_tokens[j-1]:
            alignments.append((i-1, j-1, src_chars[i-1],
tgt_tokens[j-1]))
            i -= 1
            j -= 1
        elif path[i][j] == 'up':
            i -= 1
    else:
        j -= 1

alignments.reverse()

```

```
return alignments
```

该算法优先奖励古文单字与现代文双音节词的匹配，这符合汉语演变规律，同时考虑字符在句子中的相对位置，避免跨度过大的错误对齐。识别并跳过在古文中完整出现的现代文词汇，避免误判。

A.2 多粒度正例生成

基于对齐结果，我们生成不同替换程度的正例变体：

Python

```
def process_sentence_pair(self, src_sent: str, tgt_sent: str,
                           replace_prob: float) -> Dict:
    """
    处理一对句子，生成指定替换概率的变体

    Args:
        src_sent: 古文句子
        tgt_sent: 现代文翻译
        replace_prob: 替换概率

    Returns:
        包含原文、译文、对齐文本等信息的字典
    """
    # 字符级切分古文
    src_chars = list(src_sent)

    # 分词现代文
    tgt_tokens = self.segment_target(tgt_sent)

    # 识别专有名词
    proper_nouns = self.find_proper_nouns(src_sent, tgt_tokens)

    # 执行对齐
    alignments = self.align_tokens(src_chars, tgt_tokens,
                                    proper_nouns)

    # 根据概率进行替换
    result_chars = src_chars.copy()
    replaced_alignments = []
```



```

for src_idx, tgt_idx, src_char, tgt_token in alignments:
    if random.random() < replace_prob:
        result_chars[src_idx] = tgt_token
        replaced_alignments.append({
            "src_idx": src_idx,
            "src_char": src_char,
            "tgt_idx": tgt_idx,
            "tgt_token": tgt_token
        })

return {
    "original": src_sent,
    "target": tgt_sent,
    "aligned_text": ''.join(result_chars),
    "alignments": alignments,
    "replaced_alignments": replaced_alignments,
    "replace_probability": replace_prob
}

```

通过设置不同的替换概率（30%、50%、70%），我们可以生成从古文到现代文的渐进式过渡样本。

A.3 基于 Embedding 的难负例挖掘

难负例的质量直接影响对比学习的效果。我们使用 FAISS 实现高效的相似度搜索：

```

Python
def find_hard_negatives(self,
                        query_embeddings: np.ndarray,
                        key_embeddings: np.ndarray,
                        k: int = 10,
                        exclude_self: bool = True) ->
List[List[int]]:
    """
    为每个查询找到最相似的 k 个候选作为难负例

    Args:
        query_embeddings: 查询向量矩阵
        key_embeddings: 候选向量库
        k: 返回的最相似候选数
        exclude_self: 是否排除自身

```

Returns:

每个查询的 top-k 候选索引列表

"""

构建 FAISS 索引

dimension = embeddings.shape[1]

index = faiss.IndexFlatIP(dimension) # 内积索引

归一化向量（内积等价于余弦相似度）

faiss.normalize_L2(key_embeddings)

index.add(key_embeddings.astype('float32'))

搜索最相似的候选

search_k = k + 1 if exclude_self else k

similarities, indices = index.search(
 query_embeddings.astype('float32'),
 search_k
)

处理结果

hard_negatives = []

for i, (sims, idxs) in enumerate(zip(similarities, indices)):

if exclude_self:

排除自身（通常是相似度最高的）

mask = idxs != i

filtered_idx = idxs[mask][:k]

else:

filtered_idx = idxs[:k]

hard_negatives.append(filtered_idx.tolist())

return hard_negatives

该实现通过 L2 归一化使内积等价于余弦相似度，并使用 FAISS 的平面索引实现快速最近邻搜索，并会避免将查询自身作为负例。

A.4 训练数据整合

最终的训练数据构造整合了所有技术：

Python

```
def construct_training_data(self,
```

```

        parallel_data: List[Tuple[str, str]],
        num_hard_negatives: int = 3,
        num_random_negatives: int = 1,
        bidirectional: bool = True) ->

List[Dict]:
    """
    构造完整的训练数据，整合对齐正例和难负例

    Args:
        parallel_data: 平行语料对
        num_hard_negatives: 难负例数量
        num_random_negatives: 随机负例数量
        bidirectional: 是否构造双向数据

    Returns:
        训练数据列表
    """
    training_data = []

    # 生成所有文本的 embeddings
    classical_embeddings = self.encode_texts(classical_texts)
    modern_embeddings = self.encode_texts(modern_texts)

    # 找到难负例
    hard_neg_indices = self.find_hard_negatives(
        classical_embeddings,
        modern_embeddings,
        k=num_hard_negatives * 2
    )

    for idx, (classical, modern) in enumerate(parallel_data):
        # 获取所有对齐变体作为正例
        unique_alignments =
self.extract_unique_alignments(aligned_data[idx])
        positive_examples = [modern] + unique_alignments

        # 构造负例集合
        negatives = []

        # 添加难负例

```

```

hard_negs = hard_neg_indices[idx][:num_hard_negatives]
negatives.extend([modern_texts[i] for i in hard_negs])

# 添加随机负例以增加多样性
random_indices = random.sample(
    [i for i in range(len(modern_texts)) if i != idx],
    num_random_negatives
)
negatives.extend([modern_texts[i] for i in
random_indices])

training_data.append({
    'query': classical,
    'pos': positive_examples,
    'neg': negatives
})

# 如果启用双向，同时构造现代文->古文的数据
if bidirectional:
    training_data.append({
        'query': modern,
        'pos': [classical] + unique_alignments,
        'neg': self.construct_reverse_negatives(idx)
    })

return training_data

```

该方法从前述方法构造的数据集中整合双音节词对齐法构造的正例与基于 Embedding 挖掘的难负例，构造了适合 BGE 微调的完整数据集。

附录 B：与 mRASP 方法的等价性完整证明

本附录提供双音节对齐替换技术与 mRASP (multilingual Random Aligned Substitution Pre-training) 方法在表示学习上等价性的完整数学证明。我们将证明，尽管两种方法应用于不同的领域 (跨语言 vs 跨时)，它们在优化目标和学习到的表示空间结构上具有本质的一致性。

B.1 问题设定与符号定义

定义 B.1 (文本空间)

设 $\mathcal{X}$ 为源域文本空间， $\mathcal{Y}$ 为目标域文本空间。在 mRASP 中， $\mathcal{X}$ 和 $\mathcal{Y}$ 分别对应不同语言；在我们的方法中，它们分别对应古汉语和现代汉语。

定义 B.2 (编码函数)

设 $f_\theta : \mathcal{X} \cup \mathcal{Y} \rightarrow \mathbb{R}^d$ 为参数化的编码函数，将文本映射到 d 维向量空间。我们假设 f_θ 是可微的。

定义 B.3 (对齐集合)

设 $\mathcal{A} \subset \mathcal{X} \times \mathcal{Y}$ 为语义对齐的文本单元对集合。具体地：

- 在 mRASP 中： $\mathcal{A} = (w_i^x, w_j^y) | w_i^x$ 和 w_j^y 是翻译等价的词/短语
- 在我们的方法中： $\mathcal{A} = (c_i, w_j) | c_i$ 是古文字符， w_j 是对应的现代双音节词

定义 B.4 (替换操作)

定义替换函数 $C : \mathcal{X} \rightarrow \mathcal{X} \cup \mathcal{Y}$ ，对于文本 $x = (x_1, x_2, \dots, x_n)$ ：

$C(x) = (z_1, z_2, \dots, z_n)$ ，其中 $z_i = \begin{cases} y_j & \text{if } (x_i, y_j) \in \mathcal{A} \text{ 且被选中替换 } x_i \end{cases}$

B.2 mRASP 的隐式对比学习性质

引理 B.1 mRASP 的翻译损失可以解释为一种隐式的对比学习。

证明： mRASP 优化序列到序列的翻译目标：

$$\mathcal{L}_{mRASP} = -\mathbb{E}(x, y) \sim \mathcal{D} \left[\sum_{t=1}^{|y|} \log P_\theta(y_t | C(x), y_{<t}) \right]$$

其中 $C(x)$ 是对 x 进行随机对齐替换后的结果。考虑解码器在位置 t 的输出分布：

$$P_\theta(y_t | C(x), y_{<t}) = \frac{\exp(h_t^T e(y_t))}{\sum_{v \in \mathcal{V}} \exp(h_t^T e(v))}$$

其中 h_t 是解码器在位置 t 的隐状态， $e(v)$ 是词 v 的嵌入。根据 Jozefowicz et al. (2016) 的噪声对比估计理论，当词表 $\mathcal{V}$ 很大时，softmax 可以近似为：

$$P_\theta(y_t | h_t) \approx \frac{\exp(\langle h_t, e(y_t) \rangle / \tau)}{\exp(\langle h_t, e(y_t) \rangle / \tau) + k \cdot \mathbb{E}_{v \sim P_n} [\exp(\langle h_t, e(v) \rangle / \tau)]}$$

其中 P_n 是噪声分布， k 是负采样数， τ 是温度参数。这正是对比学习的形式：最大化 h_t 与正确词 y_t 的相似度，最小化与噪声词的相似度。

B.3 对比学习目标的等价性

定理 B.1 (优化目标等价性) 在适当的条件下，mRASP 的翻译损失和我们方法的对比学习损失在表示

空间上产生相同的优化方向。

证明：

我们的方法直接优化InfoNCE损失：

$$\mathcal{L}_{contrast} = -\mathbb{E}(q, \mathcal{P}, \mathcal{N}) \left[\log \frac{\sum_{p \in \mathcal{P}} \exp(f_\theta(q)^T f_\theta(p)/\tau)}{|\mathcal{P}| \cdot (\sum_{p \in \mathcal{P}} \exp(f_\theta(q)^T f_\theta(p)/\tau) + \sum_{n \in \mathcal{N}} \exp(f_\theta(q)^T f_\theta(n)/\tau))} \right]$$

其中 q 是查询， $\mathcal{P}$ 是正例集合（包含所有对齐变体）， $\mathcal{N}$ 是负例集合。

考虑梯度更新。对于参数 θ ，InfoNCE损失的梯度为：

$$\nabla_\theta \mathcal{L}_{contrast} = -\mathbb{E} \left[\sum_{p \in \mathcal{P}} \alpha_p \nabla_\theta f_\theta(p) - \sum_{n \in \mathcal{N}} \beta_n \nabla_\theta f_\theta(n) \right]$$

其中 α_p 和 β_n 是依赖于当前相似度的权重系数。

类似地，mRASP中对于对齐对 $(x_i, y_j) \in \mathcal{A}$ ，当 x_i 被替换为 y_j 时，模型需要从 $C(x)$ （包含 y_j ）预测 y 。这要求： $f_\theta(y_j) \approx f_\theta(\text{context}(y_j \text{ in } y))$

由于 y_j 同时出现在输入和输出中，梯度更新会增强 $f_\theta(x_i)$ 和 $f_\theta(y_j)$ 的相似性。

引理B.2 对于任意对齐对 $(x_i, y_j) \in \mathcal{A}$ ，两种方法的梯度更新都满足：

$$\langle \nabla_\theta f_\theta(x_i), f_\theta(y_j) - f_\theta(x_i) \rangle > 0$$

这表明两种方法都在减小对齐文本单元之间的距离。

B.4 数据增强视角下的等价性

定理B.2（数据增强等价性） 随机对齐替换在期望意义上等价于在增广数据集上的对比学习。

证明： 设替换概率为 p ，对于每个对齐对 $(x_i, y_j) \in \mathcal{A}$ ，定义伯努利随机变量 $B_{ij} \sim \text{Bernoulli}(p)$ 。则替换后的文本为：

$$C(x) = (z_1, \dots, z_n), z_i = \begin{cases} y_j & \text{if } (x_i, y_j) \in \mathcal{A} \text{ and } B_{ij} = 1 \\ x_i & \text{otherwise} \end{cases}$$

期望损失为：

$$\mathcal{L}_{expected} = \mathbb{E} B[\mathcal{L}(C(x), y)] = \sum_{S \subseteq \mathcal{A}} P(S) \cdot \mathcal{L}(C_S(x), y)$$

其中 S 是被替换的对齐对子集， $C_S(x)$ 表示只替换 S 中的对齐对。

这等价于在以下增广数据集上优化：

$$\mathcal{D}_{aug} = (C_S(x), y) | x \in \mathcal{X}, y \in \mathcal{Y}, (x, y) \in \mathcal{D}, S \subseteq \mathcal{A}_x$$

其中 $\mathcal{A}_x$ 是 x 中所有可替换的对齐对。在 $\mathcal{D}_{aug}$ 上的对比学习将所有替换变体视为正例，从而学习到对齐不变的表示。

B.5 表示空间的收敛性分析

定理B.3（收敛性） 在温和的正则性条件下，两种方法学习到的表示空间在对齐度量下收敛到相似的结构。

证明： 定义表示空间的对齐度量：

$$\mathcal{M}(\theta) = \mathbb{E}(x, y) \in \mathcal{A} \left[\frac{\langle f_\theta(x), f_\theta(y) \rangle}{|f_\theta(x)| |f_\theta(y)|} \right]$$

设 θ_{mRASP}^t 和 $\theta_{contrast}^t$ 分别为两种方法在时刻 t 的参数。根据梯度流理论：

$$\frac{d\theta^t}{dt} = -\nabla_\theta \mathcal{L}(\theta^t)$$

由引理B.2，两种方法的梯度都指向增加对齐文本单元相似度的方向。在凸优化假设下（局部近似成立），两种方法都收敛到使 $\mathcal{M}(\theta)$ 最大化的驻点。

更严格地，假设损失函数 $\mathcal{L}$ 是 L -Lipschitz连续且 μ -强凸的，则：

$$|\theta_{mRASP}^t - \theta_{contrast}^t| \leq |\theta_{mRASP}^0 - \theta_{contrast}^0| \cdot e^{-\mu t} + \frac{2L}{\mu} \cdot \delta$$

其中：

$$\delta = \sup_{\theta} |\nabla \mathcal{L}_{mRASP}(\theta) - \nabla \mathcal{L}_{contrast}(\theta)|$$

反映了两种损失函数的差异。

因此：

$$\lim_{t \rightarrow \infty} |\mathcal{M}(\theta_{mRASP}^t) - \mathcal{M}(\theta_{contrast}^t)| \leq \frac{2L^2}{\mu^2} \cdot \delta$$

当两种方法使用相似的网络架构和训练数据时， δ 很小，从而保证了收敛表示的相似性。

B.6 计算效率优势

推论B.1 我们的方法在计算效率上优于mRASP，同时保持了表示学习的效果。

由于 mRASP需要完整的序列到序列模型进行端到端训练，计算复杂度为 $O(n^2 d)$ （考虑注意力机制）。而我们的方法直接在编码器上进行对比学习，复杂度为 $O(nd)$ 。同时，由于等价性定理，两种方法学习到相似的表示空间。

B.7 结论与讨论

本附录的证明表明双音节对齐替换技术与mRASP在表示学习的数学本质上具有优化方向等价性，都通过构建语义等价的文本变体来学习跨域不变表示。同时，我们的方法直接优化对比学习目标，避免了序列生成的计算开销，同时保持了表示质量。这种等价性揭示了跨语言和跨时对齐在表示学习中的共同原理，为其他跨域文本处理任务提供了理论指导。

附录 C：系统组件关键实现细节

C.1 YiGuaRAGServer 项目结构

YiGuaRAGServer 采用模块化的项目结构，各组件职责明确：

```
Plain Text
classical_chinese_rag/
├─ app.py                # Flask 主应用入口
├─ config.py             # 配置文件
├─ requirements.txt      # 依赖列表
├─ init_database.py      # 数据库初始化脚本
├─ api/
│   └─ __init__.py
│   └─ books.py          # 书目检索接口
│   └─ texts.py          # 文本检索接口
├─ services/
│   └─ __init__.py
│   └─ embedding_service.py # Embedding 服务
│   └─ vector_store.py    # 向量存储服务
│   └─ database_service.py # 数据库服务
│   └─ rerank_service.py  # 重排序服务
├─ models/
│   └─ __init__.py
│   └─ book.py           # 数据模型
├─ utils/
│   └─ __init__.py
│   └─ text_processor.py  # 文本处理工具
│   └─ logger.py         # 日志工具
├─ data/
│   └─ metadata/
│       └─ books.json    # 书目元数据
│   └─ raw_texts/        # 原始文本文件
└─ database/
    └─ books.db          # SQLite 数据库
    └─ chroma/           # ChromaDB 向量数据库
```

C.2 多级索引查询实现

C.2.1 书目检索 API 实现

```
Python
# api/books.py
@books_bp.route('/search', methods=['POST'])
def search_books():
    """年代与领域索引接口"""
    try:
        data = request.get_json()

        # 获取参数
        dynasties = data.get('dynasties', None)
        domains = data.get('domains', None)
        limit = data.get('limit', 10)

        # 参数验证
        if limit > 100:
            limit = 100

        logger.info(f"书目检索请求 - 朝代: {dynasties}, 领域: {domains}, 限制: {limit}")

        # 执行检索
        books = db_service.search_books(
            dynasties=dynasties,
            domains=domains,
            limit=limit
        )

        # 格式化响应
        response_books = []
        for book in books:
            response_books.append({
                'title': book['title'],
                'dynasty': book['dynasty'],
                'domain': book['domains'],
                'author': book['author'],
                'description': book['description']
            })
```

```

    })

    return jsonify({
        'status': 'success',
        'data': {
            'books': response_books,
            'total': len(response_books)
        }
    })

except Exception as e:
    logger.error(f"书目检索错误: {e}")
    return jsonify({
        'status': 'error',
        'message': str(e)
    }), 500

```

C.2.2 数据库查询逻辑

```

Python
# services/database_service.py
def search_books(self, dynasties: Optional[List[str]] = None,
                  domains: Optional[List[str]] = None,
                  limit: int = 10) -> List[Dict]:
    """根据朝代和领域搜索书目"""

    conn = sqlite3.connect(self.db_path)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    query = "SELECT * FROM books WHERE 1=1"
    params = []

    # 朝代筛选
    if dynasties:
        placeholders = ','.join(['?'] for _ in dynasties)
        query += f" AND dynasty IN ({placeholders})"
        params.extend(dynasties)

    # 领域筛选 (需要 JSON 查询)
    if domains:
        domain_conditions = []

```

```

        for domain in domains:
            domain_conditions.append("domains LIKE ?")
            params.append(f'%"{domain}"%')
        query += f" AND ({' OR '.join(domain_conditions)})"

    query += f" LIMIT ?"
    params.append(limit)

    cursor.execute(query, params)
    rows = cursor.fetchall()
    conn.close()

    results = []
    for row in rows:
        book_dict = dict(row)
        book_dict['domains'] = json.loads(book_dict['domains'])
        results.append(book_dict)

    logger.info(f"检索到 {len(results)} 本书")
    return results

```

C.3 两阶段检索实现

C.3.1 向量检索实现

```

Python
# services/vector_store.py
class VectorStore:
    """ChromaDB 向量存储服务"""

    def __init__(self, embedding_service: EmbeddingService):
        self.embedding_service = embedding_service

    # 确保目录存在
    CHROMA_DB_PATH.mkdir(parents=True, exist_ok=True)

    # 初始化 ChromaDB 客户端
    self.client = chromadb.PersistentClient(
        path=str(CHROMA_DB_PATH),
        settings=Settings(
            anonymized_telemetry=False,

```

```

        allow_reset=True
    )
)

# 创建或获取集合
self.collection = self.client.get_or_create_collection(
    name="classical_texts",
    metadata={"hnsw:space": "cosine"}
)
logger.info("向量存储初始化完成")

def search(self, queries: List[str], top_k: int = 5) ->
Dict[str, List[Dict]]:
    """向量检索"""
    results = {}

    for query in queries:
        # 编码查询
        query_embedding = self.embedding_service.encode(query)
        # 确保是一维数组
        if len(query_embedding.shape) > 1:
            query_embedding = query_embedding.squeeze()
        query_embedding_list =
query_embedding.cpu().numpy().tolist()

        # 检索
        search_results = self.collection.query(
            query_embeddings=[query_embedding_list],
            n_results=top_k,
            include=["documents", "metadatas", "distances"]
        )

        # 整理结果
        query_results = []
        if search_results['documents'] and
search_results['documents'][0]:
            for i in
range(len(search_results['documents'][0])):
                result = {
                    'text': search_results['documents'][0][i],
                    'metadata':

```

```

search_results['metadatas'][0][i],
                'score': 1 -
search_results['distances'][0][i] # 转换为相似度分数
    }
    query_results.append(result)

    results[query] = query_results

    logger.info(f"完成 {len(queries)} 个查询的检索")
    return results

```

C.3.2 重排序实现

```

Python
# services/rerank_service.py
class RerankService:
    """重排序服务"""

    def __init__(self):
        logger.info(f"加载 Rerank 模型: {RERANK_MODEL}")
        self.tokenizer =
AutoTokenizer.from_pretrained(RERANK_MODEL)
        self.model =
AutoModelForSequenceClassification.from_pretrained(RERANK_MODEL)
        self.device = 'cuda' if torch.cuda.is_available() else
'cpu'
        self.model = self.model.to(self.device)
        self.model.eval()
        logger.info(f"Rerank 模型加载完成, 使用设备: {self.device}")

    def rerank(self, query: str, documents: List[Dict], top_k: int
= 5) -> List[Dict]:
        """对检索结果进行重排序"""
        if not documents:
            return []

        pairs = [[query, doc['text']] for doc in documents]

        with torch.no_grad():
            inputs = self.tokenizer(
                pairs,

```

```

        padding=True,
        truncation=True,
        max_length=512,
        return_tensors='pt'
    ).to(self.device)

    scores = self.model(**inputs).logits.squeeze(-1)
    scores = torch.sigmoid(scores).cpu().numpy()

    # 将分数添加到文档中并排序
    for i, doc in enumerate(documents):
        doc['rerank_score'] = float(scores[i])

    # 按重排序分数降序排序
    reranked = sorted(documents, key=lambda x:
x['rerank_score'], reverse=True)

    return reranked[:top_k]

```

C.4 文言文文本处理器实现

```

Python
# utils/text_processor.py
class TextProcessor:
    """文本处理器，用于分块和预处理文言文文本"""

    @staticmethod
    def clean_text(text: str) -> str:
        """清理文本，去除多余空格和特殊字符"""
        # 去除多余空白字符
        text = re.sub(r'\s+', ' ', text)
        # 去除特殊控制字符
        text = re.sub(r'[\x00-\x1f\x7f-\x9f]', '', text)
        return text.strip()

    @staticmethod
    def split_into_chunks(text: str, max_size: int =
MAX_CHUNK_SIZE,
                                overlap: int = CHUNK_OVERLAP) ->
List[Tuple[str, int]]:
        """将文本分割成固定大小的块，返回(chunk_text, chunk_index)列
表"""

```



```

chunks = []
sentences = re.split(r'[。！？；]', text)

current_chunk = ""
chunk_index = 0

for sentence in sentences:
    sentence = sentence.strip()
    if not sentence:
        continue

    # 如果当前句子本身就超过最大长度，需要进一步分割
    if len(sentence) > max_size:
        # 按逗号分割
        sub_sentences = sentence.split(',')
        for sub_sentence in sub_sentences:
            if len(current_chunk) + len(sub_sentence) + 1
<= max_size:
                current_chunk += sub_sentence + ', '
            else:
                if current_chunk:
                    chunks.append((current_chunk.rstrip(', '), chunk_index))
                    chunk_index += 1
                    current_chunk = sub_sentence + ', '
        else:
            # 正常处理
            if len(current_chunk) + len(sentence) + 1 <=
max_size:
                current_chunk += sentence + '。 '
            else:
                if current_chunk:
                    chunks.append((current_chunk.rstrip('。 '),
chunk_index))
                    chunk_index += 1

            # 保留重叠部分
            if overlap > 0 and chunks:
                overlap_text = chunks[-1][0][-overlap:]
                current_chunk = overlap_text + sentence +
'。 '
            else:

```

```

        current_chunk = sentence + '。'

    # 处理最后一个块
    if current_chunk:
        chunks.append((current_chunk.rstrip('。'),
chunk_index))

    logger.info(f"文本被分割成 {len(chunks)} 个块")
    return chunks

```

C.5 YiGuaRAGTool 插件结构

```

Plain Text
yigua_rga_tool/
├─ manifest.yaml          # 插件元信息配置
├─ provider/
│   ├─ yigua_rga_tool.py  # 工具提供者实现
│   └─ yigua_rga_tool.yaml # 提供者配置
├─ tools/
│   ├─ book_search.py     # 书目检索工具
│   ├─ book_search.yaml   # 书目检索配置
│   ├─ text_search.py     # 文本检索工具
│   └─ text_search.yaml   # 文本检索配置
└─ main.py                # 插件入口

```

C.6 智能参数处理实现

C.6.1 工具提供者凭证验证

```

Python
# provider/yigua_rga_tool.py
class YiguaRgaToolProvider(ToolProvider):
    def _validate_credentials(self, credentials: dict[str, Any]) -
> None:
        """验证提供的凭证是否有效"""
        try:
            base_url = credentials.get("base_url", "").strip()

```

```

# 检查 base_url 是否提供
if not base_url:
    raise ToolProviderCredentialValidationError("Base
URL is required")

# 确保 URL 以 http://或 https://开头
if not base_url.startswith(("http://", "https://")):
    raise ToolProviderCredentialValidationError(
        "Base URL must start with http:// or https://"
    )

# 移除末尾的斜杠
if base_url.endswith("/"):
    base_url = base_url[:-1]

# 尝试连接到服务器验证 URL 是否可达
test_url = f"{base_url}/api/v1/books/search"

# 创建测试数据
test_data = json.dumps({
    "dynasties": ["唐"],
    "domains": ["易学"],
    "limit": 1
}).encode('utf-8')

# 创建请求
req = urllib.request.Request(
    test_url,
    data=test_data,
    headers={
        'Content-Type': 'application/json',
        'User-Agent': 'Dify-Plugin/1.0'
    },
    method='POST'
)

# 发送请求（超时设置为 5 秒）
try:
    with urllib.request.urlopen(req, timeout=5) as
response:

```

问题

```
        pass # 只需要确认服务器响应即可
    except urllib.error.HTTPError as e:
        # 如果返回 400-499 错误, 说明服务器存在但请求格式可能有

        if 400 <= e.code < 500:
            pass
        else:
            raise ToolProviderCredentialValidationError(
                f"Server returned error: {e.code}"
            )
    except urllib.error.URLError as e:
        raise ToolProviderCredentialValidationError(
            f"Cannot connect to server: {str(e)}"
        )

except ToolProviderCredentialValidationError:
    raise
except Exception as e:
    raise ToolProviderCredentialValidationError(
        f"Credential validation failed: {str(e)}"
    )
```

C.6.2 参数解析与处理

Python

```
# tools/book_search.py
def _invoke(self, tool_parameters: dict[str, Any]) ->
Generator[ToolInvokeMessage, None, None]:
    """调用书目检索 API"""

    # 获取参数
    dynasties_str = tool_parameters.get("dynasties", "")
    domains_str = tool_parameters.get("domains", "")
    limit = tool_parameters.get("limit", 10)

    # 处理朝代和领域参数
    dynasties = []
    domains = []

    if dynasties_str:
        # 分割并清理朝代列表
```

```

        dynasties = [d.strip() for d in dynasties_str.split(",")
if d.strip()]

    if domains_str:
        # 分割并清理领域列表
        domains = [d.strip() for d in domains_str.split(",") if
d.strip()]

    # 构建请求数据
    request_data = {}
    if dynasties:
        request_data["dynasties"] = dynasties
    if domains:
        request_data["domains"] = domains
    request_data["limit"] = int(limit)

```

C.7 多格式输出处理实现

```

Python
# tools/book_search.py
def _invoke(self, tool_parameters: dict[str, Any]) ->
Generator[ToolInvokeMessage, None, None]:
    """处理检索结果并生成多格式输出"""

    # ... 前置处理代码 ...

    with urllib.request.urlopen(req, timeout=30) as response:
        response_data = response.read().decode('utf-8')
        result = json.loads(response_data)

        # 解析结果
        if isinstance(result, dict) and "books" in result:
            books = result["books"]
        elif isinstance(result, list):
            books = result
        else:
            books = []

        # 生成用户友好的文本消息
        if books:
            message = f"找到 {len(books)} 本相关书籍 : \n\n"

```

```

        for i, book in enumerate(books, 1):
            title = book.get("title", "未知书名")
            dynasty = book.get("dynasty", "未知朝代")
            domain = book.get("domain", "未知领域")
            author = book.get("author", "未知作者")

            message += f"{i}. 《{title}》\n"
            message += f"    朝代 : {dynasty}\n"
            message += f"    领域 : {domain}\n"
            message += f"    作者 : {author}\n\n"

        yield self.create_text_message(message)
    else:
        yield self.create_text_message("未找到符合条件的书籍。")

# 返回 JSON 格式的结果
yield self.create_json_message({
    "books": books,
    "count": len(books)
})

# 为工作流创建变量
if books:
    # 提取书名列表, 供后续 text_search 使用
    book_titles = [book.get("title", "") for book in books
if book.get("title")]
    yield self.create_variable_message("book_titles",
book_titles)

```

C.8 错误处理实现

```

Python
# tools/text_search.py
def _invoke(self, tool_parameters: dict[str, Any]) ->
Generator[ToolInvokeMessage, None, None]:
    """调用文本检索 API, 包含完整的错误处理"""
    try:
        # 获取 base_url

```

```

        base_url = self.runtime.credentials.get("base_url",
        "").strip()
        if base_url.endswith("/"):
            base_url = base_url[:-1]

        # 验证必需参数
        title_str = tool_parameters.get("title", "")
        queries_str = tool_parameters.get("queries", "")

        if not title_str:
            yield self.create_text_message("错误：请提供要检索的书
名。")
            return

        if not queries_str:
            yield self.create_text_message("错误：请提供检索关键词。
")
            return

        # ... 参数处理代码 ...

        try:
            with urllib.request.urlopen(req, timeout=30) as
response:
                # ... 正常处理逻辑 ...

        except urllib.error.HTTPError as e:
            error_body = e.read().decode('utf-8')
            yield self.create_text_message(f"API 请求失败 (HTTP
{e.code}): {error_body}")
        except urllib.error.URLError as e:
            yield self.create_text_message(f"网络连接错误:
{str(e)}")
        except json.JSONDecodeError as e:
            yield self.create_text_message(f"响应解析错误:
{str(e)}")

        except Exception as e:
            yield self.create_text_message(f"文本检索错误: {str(e)}")

```

C.9 YiGuaAgent 典型交互示例

C.9.1 单轮检索示例

Plain Text

用户：什么是六爻？

Agent 思考：用户询问六爻的概念，这是易学卜卦领域的基础知识，需要检索相关书籍。

Action 1: book_search

参数：{
 "dynasties": "", # 不限定朝代
 "domains": "卜卦,易学",
 "limit": 5
}

Observation 1: 找到 5 本相关书籍：

1. 《周易》- 战国 - 卜卦
2. 《增删卜易》- 清 - 卜卦
3. 《火珠林》- 明 - 卜卦
- ...

Action 2: text_search

参数：{
 "title": "周易,增删卜易",
 "queries": "六爻,爻辞,变爻",
 "top_k": 5
}

Observation 2: 找到相关文本 5 条...

最终回答：六爻是《周易》占卜体系中的核心概念。根据《周易》原文："爻者，言乎变者也"，每一卦由六条爻组成，从下到上分别称为初爻、二爻、三爻、四爻、五爻、上爻...

C.9.2 多轮深入检索示例

Plain Text

用户：请详细解释乾卦的含义和应用

Agent 思考：这是一个需要深入解释的问题，需要分步骤检索：

1. 先找到包含乾卦的主要典籍
2. 检索乾卦的卦辞和爻辞
3. 检索历代易学家的注解

Action 1: book_search

参数：{

"dynasties": "战国,汉,宋", # 重要的易学发展时期

"domains": "易学,卜卦",

"limit": 10

}

Observation 1: 找到《周易》《周易正义》《周易程氏传》等 10 本书籍

Action 2: text_search

参数：{

"title": "周易",

"queries": "乾卦,元亨利贞,天行健",

"top_k": 5

}

Observation 2: 检索到乾卦原文及基础解释

Action 3: text_search

参数：{

"title": "周易正义,周易程氏传",

"queries": "乾卦注解,乾为天",

"top_k": 3

}

Observation 3: 检索到历代注解内容

最终回答：

乾卦是《周易》六十四卦之首，象征天、刚健、君子之德。

【原文解析】

根据《周易》原文："乾：元亨利贞。"这四个字概括了乾卦的核心含义...

【历代注解】

1. 孔颖达《周易正义》认为："乾者健也，言天之体以健为用..."
2. 程颐《周易程氏传》解释："乾，天也。天者天之形体，乾者天之性情..."

【实际应用】

在占卜中，得乾卦主吉，象征事业顺利、君子得位...

附录 D：成员分工

姓名	学号	工作
王殿云	2022211733	- 古汉语处理、Embedding、ReAct 技术调研 - 易卦 AI 小程序搭建与部署 - Dify 插件开发 - BGE 模型微调框架搭建 - 基于双音节对齐替换技术的 BGE 微调方法设计与训练 - 开题与结题报告撰写 - 开题与结题汇报
王博涵	2022211735	- Embedding 技术调研 - 易卦 AI 小程序部署 - Dify 部署与集成 - 古今平行语料数据集构建 - BGE 模型微调框架搭建 - 基于双音节对齐替换技术的 BGE 微调方法训练与分析 - 开题与中期报告撰写
王艺栋	2022211799	- Rerank 技术调研 - 易卦 AI 小程序部署 - Dify 部署与集成 - 易学书籍数据收集、清理与索引构建 - RAG 框架搭建 - 基于双音节对齐替换技术的 BGE 微调方法训练与分析 - 开题与中期报告撰写